

Reviewer Pack — Coarse-dimensional KRW direct sum: asdim is sub-additive und...

Entry f685d342f4c0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 16:44:42 UTC

Coarse-dimensional KRW direct sum: asdim is sub-additive under block composition

Entry ID: f685d342f4c0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 16:44:42 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Karchmer-Wigderson (CG-KW) framework; coarse geometry / asymptotic-dimension theory (Gromov, Bell-Dranishnikov) **Field B** (complexity object): Boolean formula-depth complexity; the Karchmer-Raz-Wigderson (KRW) direct-sum conjecture for NC^1 lower bounds

Statement:

For all Boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$ and $g: \{0,1\}^m \rightarrow \{0,1\}$, let (X_f, d_f) and (X_g, d_g) denote their KW-metric spaces and let $(X_{f \circ g}, d_{f \circ g})$ be the coarse product encoding the block composition $f \circ g$. Then there exists a universal constant $C \leq 2$ such that $\text{asdim}(X_{f \circ g}) \geq \text{asdim}(X_f) + \text{asdim}(X_g) - C$. Equivalently, certified by the asymptotic-dimension-witness primitive: if χ_f, χ_g are optimal (k_f, R_f) - and (k_g, R_g) -witnesses for X_f, X_g respectively, then no $(k_f + k_g - C - 1, R)$ -witness exists for $X_{f \circ g}$ for any finite R . Combined with axiom A1, this yields the KRW-style implication $d(f \circ g) \geq \Omega(d(f)) + \Omega(d(g)) - O(1)$.

Rationale (proposer's reasoning):

This sub-conjecture tests axiom A2 (Composition sub-additivity) directly using the framework's coarse_product primitive. It mirrors the classical Bell-Dranishnikov inequality $\text{asdim}(X \times Y) \leq \text{asdim}(X) + \text{asdim}(Y)$ for metric products, asserting that the matching lower bound also holds for the depth-adjusted KW-product metric $d_{f \circ g}$. The mechanism is structural: a controlled cover of $X_{f \circ g}$ of multiplicity m must, when restricted to fibers, refine into controlled covers of X_f and X_g , so the multiplicity exponents add. Verifying A2 on small instances closes the gap between κ -additivity and KRW, since Andreev-style amplification then bootstraps κ -bounds through composition (Axiom A5).

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time:)

Framework membership: framework fw_85a254b4a0, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7a6eac3d592428f8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 36 (f,g) pairs from {AND_2,OR_2,XOR_2,MAJ_3,MUX_3,AND_3}, compute slack = asdim($X_{\{f \circ g\}}$) - asdim(X_f) - asdim(X_g). Support requires $\geq 95\%$ of pairs with slack ≥ -2 (C=2) AND min slack ≥ -2 AND slack = 0 saturation on both AND_2 $\circ$ AND_2 and XOR_2 $\circ$ XOR_2. Single deterministic run (no seeds; enumeration).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.86	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.86	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Karchmer-Wigderson asymptotic dimension coarse geometry formula depth - KRW composition conjecture direct sum formula complexity lower bound - asymptotic dimension subadditive product Karchmer Raz Wigderson Boolean

Top relevant hits considered: - [http://arxiv.org/abs/2107.05128v2] Karchmer-Wigderson Games for Hazard-free Computation - [http://arxiv.org/abs/2002.07444v1] Multiparty Karchmer-Wigderson Games and Threshold Circuits - [http://arxiv.org/abs/0711.0446v1] Asymptotic Properties of Hilbert Geometry - [http://arxiv.org/abs/2306.00615v5] Toward Better Depth Lower Bounds: A KRW-like theorem for Strong Composition - [http://arxiv.org/abs/2307.07605v1] First-order Methods for Affinely Constrained Composite Non-convex Non-smooth Problems: Lower Complexity Bound and Near-o - [http://arxiv.org/abs/cs/0111062v2] One-way communication complexity and the Neciporuk lower bound on formula size - [http://arxiv.org/abs/2405.01069v1] Oriented Ramsey numbers of graded digraphs - [http://arxiv.org/abs/1504.05012v1] Polynomials vanishing on Cartesian products: The Elekes-Szabó Theorem revisited

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import itertools
import math
import json

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def AND_2(x, y):
        return x and y
```

```

def OR_2(x, y):
    return x or y

def XOR_2(x, y):
    return x != y

def MAJ_3(x, y, z):
    return (x + y + z) >= 2

def MUX_3(x, y, z):
    return y if x else z

def AND_3(x, y, z):
    return x and y and z

functions = [AND_2, OR_2, XOR_2, MAJ_3, MUX_3, AND_3]

def d_f(f, x, y):
    return math.ceil(math.log2(sum(1 for i in range(len(x)) if x[i] != y[i])))

def asdim(X_f, d_f):
    n = len(X_f)
    k = 0
    while True:
        found = False
        for R in range(1, n + 1):
            colors = [None] * n
            used_colors = set()
            for i in range(n):
                if colors[i] is None:
                    color = len(used_colors)
                    used_colors.add(color)
                    queue = [i]
                    while queue:
                        current = queue.pop()
                        if colors[current] is not None:
                            continue
                        colors[current] = color
                        for j in range(n):
                            if d_f(X_f[i], X_f[j]) <= R and colors[j] is None:
                                queue.append(j)
            if len(used_colors) == k + 1:
                found = True
                break
        if not found:
            return k
        k += 1

def coarse_product(X_f, d_f, X_g, d_g):
    n = len(X_f)
    m = len(X_g)
    X_fg = [tuple(x) for x in itertools.product(X_f, X_g)]
    d_fg = lambda (x1, y1), (x2, y2): max(d_f(x1, x2), d_g(y1, y2))
    return X_fg, d_fg

```

```

results = []
for f in functions:
    for g in functions:
        X_f = [tuple(f(*x) for x in itertools.product([0, 1], repeat=len(f.__code__.co_varnames)))
        X_g = [tuple(g(*x) for x in itertools.product([0, 1], repeat=len(g.__code__.co_varnames)))
        X_fg, d_fg = coarse_product(X_f, d_f, X_g, d_g)
        asdim_fg = asdim(X_fg, d_fg)
        asdim_f = asdim(X_f, d_f)
        asdim_g = asdim(X_g, d_g)
        slack = asdim_fg - asdim_f - asdim_g
        results.append((f.__name__, g.__name__, slack))

total_slack = sum(slack for _, _, slack in results)
min_slack = min(slack for _, _, slack in results)
support_fraction = sum(1 for _, _, slack in results if slack >= -2) / len(results)

return {
    "metric_name": "slack",
    "metric_value": total_slack,
    "instances_tested": len(results),
    "conjecture_holds": support_fraction >= 0.95 and min_slack == -2 and any(slack == 0 for _, _, slack in results),
    "counterexample": "" if support_fraction >= 0.95 and min_slack == -2 and any(slack == 0 for _, _, slack in results) else None
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_slack = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_slack) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.95 and min(r["metric_value"] for r in results) == -2 and any(r["conjecture_holds"]):
        print(f"RESULT: SUPPORTED mean={mean_slack} std={std_dev} support_fraction={support_fraction}")
    elif min(r["metric_value"] for r in results) < -2:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["conjecture_holds"] == False)
        print(f"RESULT: FALSIFIED counterexample=\"min_slack < -2\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction} min_slack={min(r['metric_value'] for r in results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2678b93a.py", line 158, in <module>
 result = run_trial(seed)

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2678b93a.py", line 110, in run_trial
    d_f_val = max(d_f(f, x, y) for x, y in itertools.combinations(X_f, 2))
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2678b93a.py", line 110, in <genexpr>
    d_f_val = max(d_f(f, x, y) for x, y in itertools.combinations(X_f, 2))

```

NameError: name 'f' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a NameError ('f' is not defined') before producing any data, so the pre-registered support condition could not be evaluated. Per Rule 2, a crash yields INCONCLUSIVE. | next: Fix the bug in run_trial (line 110) by passing f as an argument to d_f within the generator expression, then re-run the deterministic enumeration over all 36 (f,g) pairs.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	7031
2	novelty	claude_max	opus	0	0	5708
3	novelty	claude_max	opus	0	0	9646
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25219
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14880
6	judge	claude_max	opus	0	0	5666

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 68150 ms total latency. Provider mix: {'claude max': 4, 'ollama remote': 2}

(full prompt+response transcripts available in [research/audit/f685d342f4c0.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f685d342f4c0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f685d342f4c0.tar.gz (if generated)